VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Music Streaming Database

A9512- Database Management System Laboratory

Submitted by

Shaik Jasmin Haneef	25881A0514
Kommu Kaveri	25881A0515
Patolla Sathwika	25881A0553

Course Instructor

Ms. Shaik Shahanaz
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to JNTUH,
Approved by AICTE, Accredited by NAAC with A++ Grade
Kacharam, Shamshabad, Hyderabad-501 218,

Telangana, India.

Vibrant Vardhaman

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Music Streaming Database"** is carried out by **"Shaik Jasmin Haneef (25881A0514), Patolla Sathwika (25881A0553), Kommu Kaveri (25881A0515)"** towards **A9512 – DataBase Management System Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology in Computer Science and Engineering** during the Academic year 2026–27.

Course Facilitator	Academic Head	Head of the Department
Ms. Shaik Shahanaz	**Dr. R. Karthikeyan**	**Prof. Y. Vijayalata**
Assistant Professor, CSE	Professor, CSE	HoD, CSE

Contents

1 Introduction 5
2 Problem Analysis 5
2.1 Problem Statement 5
2.2 Users and Assumptions 5
2.3 Functional Requirements 6
3 ER Model 7
3.1 Entities and Attributes 7
3.2 Relationships 7
3.3 Cardinality and Participation 8
3.4 ER Diagram 9
4 Relational Model 9
4.1 Relational Schema 9
4.2 Primary Keys and Foreign Keys 10
5 Normalization 11
5.1 Functional Dependencies 11
5.2 First Normal Form (1NF) 12
5.3 Second Normal Form (2NF) 12
5.4 Third Normal Form (3NF) 13
6 SQL DDL Commands 13
6.1 Table Creation 14
6.2 Constraints 15
6.3 ALTER Operation 16
6.4 DROP Operation 16
7 SQL DML Commands 16
7.1 INSERT Operation 17
7.2 UPDATE Operation 18
7.3 DELETE Operation 18
8 SQL Queries 18
8.1 Basic SELECT Queries 18

8.2 Joins . . . 19
8.3 Aggregate Functions . . . 19
8.4 GROUP BY and HAVING . . . 20
8.5 Subqueries . . . 21
8.6 Views . . . 21
9 Transactions and Concurrency . . . 22
9.1 Transaction Management . . . 22
9.2 COMMIT, ROLLBACK and SAVEPOINT . . . 23
9.3 Concurrency Control . . . 23
10 PL/SQL Implementation . . . 24
10.1 Functions . . . 24
10.2 Stored Procedures . . . 25
10.3 Cursors . . . 25
10.4 Triggers . . . 26
11 File Organization and Indexing . . . 27
11.1 File Organization . . . 27
11.2 ISAM . . . 27
11.3 B+ Tree Indexing . . . 27
12 Testing . . . 28
12.1 Test Cases . . . 28
12.2 Normal and Boundary Cases . . . 29
12.3 Invalid Input Testing . . . 29
13 Reports . . . 30
13.1 Artist Report . . . 30
13.2 Popular Songs Report . . . 30
13.3 Genre-Wise Report . . . 30
13.4 User Listening Statistics . . . 31
14 Results and Discussion . . . 31
15 Conclusion . . . 32
16 SDG Mapping . . . 32
17 Future Scope . . . 33
Bibliography . . . 34

Abstarct

The Music Streaming Database Management System is designed to organize and manage information related to users, songs, artists, albums, genres, playlists, and listening history. The system provides a structured database for maintaining music catalog information and recording user interactions such as playlist creation, song likes, artist follows, and listening activity.

The database is designed using relational database concepts, including entities, relationships, primary keys, foreign keys, integrity constraints, and normalization. Oracle SQL is used to create tables, manage records, and retrieve information through queries, joins, aggregate functions, subqueries, and views. PL/SQL is used to implement selected database operations through procedures, functions, cursors, and triggers.

An additional feature of the system is SQL-based song recommendation. By analyzing a user's listening history, the database can identify songs associated with previously played genres or artists and retrieve suitable recommendations. The system can also generate reports about popular songs, frequently played genres, and listening patterns.

The project demonstrates the practical application of database management concepts in a music streaming scenario, with an emphasis on data integrity, reduced redundancy, efficient retrieval, and maintainable database design.

Keywords: Music Streaming, Database Management System, SQL, Users, Artists, Albums, Songs, Playlists, Listening History, Relational Database

1 Introduction

Music streaming platforms allow users to search for songs, explore artists and albums, create playlists, and maintain their listening preferences. Managing these activities requires a structured database that can store music-related information and user interactions efficiently.

The Music Streaming Database Management System is designed to manage information about users, artists, albums, songs, genres, playlists, and listening history. The database establishes relationships between these entities using primary keys and foreign keys, ensuring data consistency and reducing redundancy.

The system supports essential operations such as song searching, playlist creation, and management of music catalogue information. It also maintains users' listening history, liked songs, and followed artists. This information can be used to generate artist-related reports, identify popular songs, and calculate listening statistics.

Oracle SQL is used to create database tables, insert and update records, and retrieve information through queries, joins, aggregate functions, and views. PL/SQL is used to implement selected database operations. Listening-history-based SQL queries are also included to identify potential song recommendations based on users' previously played songs and preferred genres.

The project demonstrates the practical application of database management concepts in a music streaming environment. It focuses on organized data storage, efficient information retrieval, relational database design, and the use of SQL and PL/SQL to support music catalogue management and user activity analysis.

2 Problem Analysis

2.1 Problem Statement

The Music Streaming Database Management System is designed to organize and manage the information required by a music streaming platform. The system maintains records of users, songs, artists, albums, genres, playlists, and listening history. It supports playlist management, song likes, artist follows, and music recommendations based on listening activity.

The database uses a relational structure to reduce data redundancy and maintain consistency. Oracle SQL and PL/SQL are used to manage records, retrieve information, and automate selected database operations. The system also supports analytical queries for identifying popular songs, frequently played genres, and user listening patterns.

2.2 Users and Assumptions

The system supports the following users:

- **Administrator:** Manages users, songs, artists, albums, and genres.
- **Registered User:** Creates playlists, likes songs, follows artists, and maintains listening history.
- **Report Viewer:** Retrieves music statistics, popular songs, and recommendation results.

The following assumptions are considered during database design:

1. Each user, song, artist, album, genre, and playlist has a unique identifier.
2. Each user has a unique email address.
3. Each song belongs to one album and one primary genre.
4. A song can be associated with multiple artists, and an artist can contribute to multiple songs.
5. A user can create multiple playlists, and each playlist can contain multiple songs.
6. Listening history records the user, song, and time of each listening event.
7. A user can like a song and follow an artist only once at a time.
8. Recommendations are generated using SQL queries based on listening history and shared genres or artists.
9. Subscription management, payment processing, and actual audio streaming are outside the scope of this project.

2.3 Functional Requirements

The system must support the following operations:

1. **User Management:** Register users and maintain their profile information.
2. **Song Management:** Store and retrieve song details, including title, duration, release date, album, and genre.
3. **Artist Management:** Maintain artist information and associate artists with songs.
4. **Album Management:** Store album details and their associated artists.
5. **Genre Management:** Organize songs according to their genres.
6. **Playlist Management:** Create playlists and add or remove songs.
7. **Listening History:** Record users' listening activity with timestamps.
8. **Like Management:** Allow users to like songs and retrieve their liked songs.
9. **Artist Following:** Allow users to follow artists and view their followed artists.
10. **Music Search:** Search for songs by title, artist, album, or genre.
11. **Song Recommendations:** Recommend songs based on previously played songs, their genres, and associated artists.
12. **Statistical Reports:** Generate reports showing popular songs, artist popularity, and listening patterns.
13. **Database Integrity:** Maintain consistency through primary keys, foreign keys, and other constraints.
14. **Database Operations:** Demonstrate SQL queries, views, transactions, and PL/SQL procedures, functions, cursors, and triggers.

3 ER Model

The Entity-Relationship (ER) model represents the entities, attributes, and relationships of the Music Streaming Database Management System. It provides a conceptual representation of how users, artists, albums, songs, genres, playlists, and listening history are connected. This model serves as the foundation for designing the relational database.

3.1 Entities and Attributes

The main entities and their attributes are presented in Table 3.1. Primary keys uniquely identify records, while foreign keys connect related entities.

Table 1: Main Entities and Attributes

Entity	**Key Attribute**	**Other Attributes**
Music_User	User_ID (PK)	User_Name, Email, Join_Date
Artist	Artist_ID (PK)	Artist_Name, Country
Album	Album_ID (PK)	Album_Title, Release_Date, Artist_ID (FK)
Genre	Genre_ID (PK)	Genre_Name
Song	Song_ID (PK)	Song_Title, Duration, Release_Date, Album_ID (FK), Genre_ID (FK), Language
Song_Artist	Song_ID + Artist_ID (PK/FK)	Associates songs with artists
Playlist	Playlist_ID (PK)	Playlist_Name, Created_Date, User_ID (FK)
Playlist_Song	Playlist_ID + Song_ID (PK/FK)	Added_Date
Listening_History	History_ID (PK)	User_ID (FK), Song_ID (FK), Listen_Time
Song_Like	User_ID + Song_ID (PK/FK)	Like_Date
Artist_Follow	User_ID + Artist_ID (PK/FK)	Follow_Date

3.2 Relationships

The relationships between the main entities are listed in Table 3.2.

Table 2: Relationships Between Entities

Relationship	**Description**
Artist–Album	An artist releases albums.
Album–Song	An album contains songs.
Genre–Song	A genre classifies songs.
Song–Artist	Artists perform or collaborate on songs.
Music User–Playlist	A user creates playlists.
Playlist–Song	Playlists contain songs.
Music User–Listening History	Users generate listening history records.
Song–Listening History	Songs are recorded in listening history.
Music User–Song Like	Users like songs.
Music User–Artist Follow	Users follow artists.

3.3 Cardinality and Participation

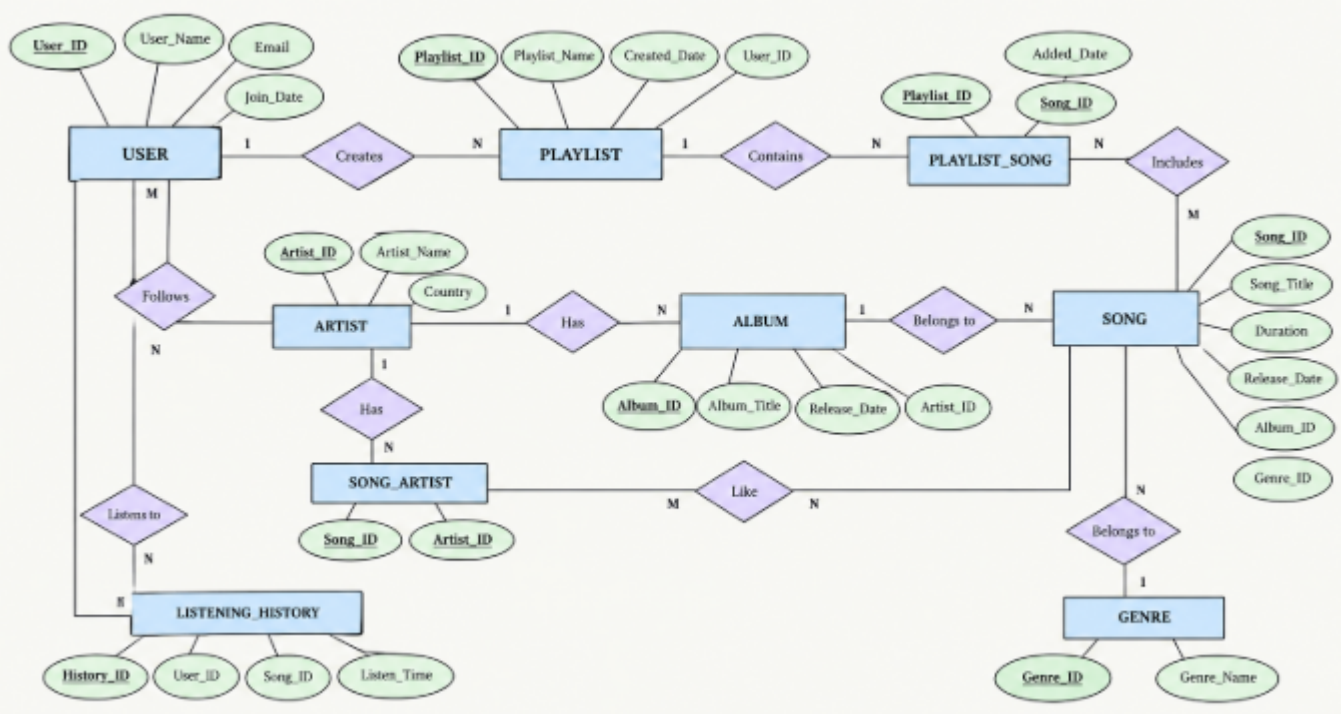
Cardinality specifies the number of records that can participate in a relationship. Participation indicates whether an entity's involvement in a relationship is mandatory or optional. Table 3.3 summarizes these properties.

Table 3: Cardinality and Participation

Relationship	**Cardinality**	**Participation**
Artist–Album	1:N	An album requires an artist; an artist may have no albums.
Album–Song	1:N	A song belongs to an album; an album may contain multiple songs.
Genre–Song	1:N	A song requires a genre; a genre may classify multiple songs.
Song–Artist	M:N	A song has at least one associated artist; an artist may have multiple songs.
Music User–Playlist	1:N	A playlist requires a user; a user may create multiple playlists.
Playlist–Song	M:N	A playlist may contain multiple songs, and a song may appear in multiple playlists.
Music User–Listening History	1:N	Each history record requires a user; a user may have multiple records.
Song–Listening History	1:N	Each history record refers to a song; a song may appear in multiple records.
Music User–Song Like	M:N	Like records require a user and a song; either may have no associated likes.
Music User–Artist Follow	M:N	Follow records require a user and an artist; either may have no follow records.

3.4 ER Diagram

The ER diagram illustrates the entities, attributes, and relationships of the Music Streaming Database Management System. Entities are represented by rectangles, attributes by ovals, and relationships by diamonds. Cardinality labels indicate one-to-many and many-to-many relationships.



The diagram includes the main entities and associative entities such as Song_Artist, Playlist_Song, Song_Like, and Artist_Follow. It also represents Listening_History as a separate entity to record repeated listening events by users.

4 Relational Model

The relational model represents the Music Streaming Database using tables connected through primary keys and foreign keys. Each table stores information about a specific entity or relationship. This structure helps maintain data integrity, minimize redundancy, and retrieve related information efficiently.

4.1 Relational Schema

The relational schema of the Music Streaming Database consists of the following relations:

Table 4: Relational Schema

Relation	**Attributes**
Music_User	User_ID, User_Name, Email, Join_Date
Artist	Artist_ID, Artist_Name, Country
Album	Album_ID, Album_Title, Release_Date, Artist_ID
Genre	Genre_ID, Genre_Name
Song	Song_ID, Song_Title, Duration, Release_Date, Album_ID, Genre_ID, Language
Song_Artist	Song_ID, Artist_ID
Playlist	Playlist_ID, Playlist_Name, Created_Date, User_ID
Playlist_Song	Playlist_ID, Song_ID, Added_Date
Listening_History	History_ID, User_ID, Song_ID, Listen_Time
Song_Like	User_ID, Song_ID, Like_Date
Artist_Follow	User_ID, Artist_ID, Follow_Date

The schema includes the main entities and associative tables required to represent multiple artists per song, multiple songs per playlist, user listening activity, liked songs, and followed artists. The `Language` attribute in the Song relation stores the language of each song.

4.2 Primary Keys and Foreign Keys

Primary keys uniquely identify records in a relation, while foreign keys establish relationships between related tables. Composite primary keys are used in associative tables to prevent duplicate relationships.

Table 5: Primary Keys and Foreign Keys

Relation	**Primary Key**	**Foreign Key(s)**
Music_User	User_ID	None
Artist	Artist_ID	None
Album	Album_ID	Artist_ID references Artist(Artist_ID)
Genre	Genre_ID	None
Song	Song_ID	Album_ID references Album(Album_ID); Genre_ID references Genre(Genre_ID)
Song_Artist	Song_ID, Artist_ID	Song_ID references Song(Song_ID); Artist_ID references Artist(Artist_ID)
Playlist	Playlist_ID	User_ID references Music_User(User_ID)
Playlist_Song	Playlist_ID, Song_ID	Playlist_ID references Playlist(Playlist_ID); Song_ID references Song(Song_ID)
Listening_History	History_ID	User_ID references Music_User(User_ID); Song_ID references Song(Song_ID)
Song_Like	User_ID, Song_ID	User_ID references Music_User(User_ID); Song_ID references Song(Song_ID)
Artist_Follow	User_ID, Artist_ID	User_ID references Music_User(User_ID); Artist_ID references Artist(Artist_ID)

The primary and foreign key constraints ensure entity integrity and referential integrity. For

example, a playlist must belong to an existing user, and a listening-history record must refer to an existing user and song. Composite keys in Song_Artist, Playlist_Song, Song_Like, and Artist_Follow prevent duplicate associations. Listening_History uses a separate History_ID so that the same user can listen to the same song multiple times.

5 Normalization

Normalization is the process of organizing data in a database to reduce redundancy and prevent insertion, deletion, and update anomalies. In the Music Streaming Database Management System, normalization is applied to organize user information, music catalog details, playlists, and listening records into related tables. The database is designed to satisfy First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

5.1 Functional Dependencies

A functional dependency describes a relationship between attributes in which one attribute or a set of attributes determines another attribute. Functional dependencies help identify keys and organize the database into normalized relations.

The major functional dependencies in the system are shown below.

Table 6: Functional Dependencies

Relation	**Functional Dependency**
User	User_ID → User_Name, Email, Join_Date
Artist	Artist_ID → Artist_Name, Country
Album	Album_ID → Album_Title, Release_Date, Artist_ID
Genre	Genre_ID → Genre_Name
Song	Song_ID → Song_Title, Duration, Release_Date, Album_ID, Genre_ID
Playlist	Playlist_ID → Playlist_Name, Created_Date, User_ID
Listening_History	History_ID → User_ID, Song_ID, Listen_Time
Song_Artist	(Song_ID, Artist_ID) uniquely identifies each association
Playlist_Song	(Playlist_ID, Song_ID) → Added_Date
Song_Like	(User_ID, Song_ID) → Like_Date
Artist_Follow	(User_ID, Artist_ID) → Follow_Date

These dependencies indicate that each primary key determines the non-key attributes of its relation. The junction tables use composite keys to identify the association between the corresponding entities.

5.2 First Normal Form (1NF)

A relation is in First Normal Form when every attribute contains a single, atomic value and there are no repeating groups or multivalued attributes in a single field.

In the Music Streaming Database, each user has a separate record in the User table, and each song has a separate record in the Song table. Song titles, durations, release dates, and genre identifiers are stored in individual columns. Multiple artists associated with a song are stored as separate records in the Song_Artist table instead of storing multiple artist names in one field.

For example, the following unnormalized structure stores multiple songs in a single playlist field.

Table 7: Example of an Unnormalized Playlist

Playlist_ID	**Playlist_Name**	**Songs**
101	Chill Mix	Song A, Song B, Song C

This structure violates 1NF because the Songs field contains multiple values. It is converted into atomic records using the Playlist and Playlist_Song tables.

Table 8: Playlist Songs in First Normal Form

Playlist_ID	**Playlist_Name**	**Song_ID**
101	Chill Mix	201
101	Chill Mix	202
101	Chill Mix	203

Each row now represents one playlist-song association. In the final design, playlist details are stored once in Playlist, while individual song associations are maintained in Playlist_Song.

5.3 Second Normal Form (2NF)

A relation is in Second Normal Form when it is in 1NF and every non-key attribute depends on the entire candidate key, rather than only a part of a composite key.

Consider a combined playlist-song relation containing the playlist name and song title.

Table 9: Example of a Relation with Partial Dependencies

Playlist_ID	**Song_ID**	**Playlist_Name**	**Song_Title**
101	201	Chill Mix	Song A
101	202	Chill Mix	Song B
102	201	Favourites	Song A

The composite key is (Playlist_ID, Song_ID). However, Playlist_Name depends only on Playlist_ID, and Song_Title depends only on Song_ID. These are partial dependencies, so the combined relation does not satisfy 2NF.

To remove these partial dependencies, the information is separated into the following relations:

- Playlist(Playlist_ID, Playlist_Name, Created_Date, User_ID)
- Song(Song_ID, Song_Title, Duration, Release_Date, Album_ID, Genre_ID)
- Playlist_Song(Playlist_ID, Song_ID, Added_Date)

In Playlist_Song, Added_Date depends on the complete composite key. The playlist name and song title are stored in their respective tables. Therefore, the design removes the illustrated partial dependencies.

5.4 Third Normal Form (3NF)

A relation is in Third Normal Form when it is in 2NF and contains no transitive dependencies of non-key attributes on a candidate key. Non-key attributes should depend on the key, the whole key, and nothing but the key.

Consider storing genre details directly in the Song table.

Table 10: Example of a Transitive Dependency

Song_ID	**Song_Title**	**Genre_Name**
201	Song A	Pop
202	Song B	Rock
203	Song C	Pop

If the design stores Genre_ID and Genre_Name together in the Song table, the dependencies are:

$$Song_ID \rightarrow Genre_ID$$

$$Genre_ID \rightarrow Genre_Name$$

This creates a transitive dependency from Song_ID to Genre_Name through Genre_ID. To remove it, genre details are stored separately in the Genre table, while the Song table stores only Genre_ID as a foreign key.

The resulting relations are:

- Song(Song_ID, Song_Title, Duration, Release_Date, Album_ID, Genre_ID)
- Genre(Genre_ID, Genre_Name)

Now, Genre_Name is determined by Genre_ID in the Genre table. This avoids repeated genre names in the Song table and simplifies updates to genre information.

Normalization Summary

The database design applies the three normal forms as follows:

Normalization reduces unnecessary data duplication and helps maintain consistency across the Music Streaming Database. It also supports reliable updates, deletion, and retrieval operations.

6 SQL DDL Commands

Data Definition Language (DDL) commands are used to create, modify, and remove database objects. In this project, Oracle SQL DDL commands define the tables required to manage users, artists, albums, songs, genres, playlists, and listening activities.

Table 11: Summary of Normalization

Normal Form	**Requirement**	**Application**
1NF	Atomic values and no repeating groups	Each song, artist association, and playlist-song association is stored separately
2NF	No partial dependency on a composite key	Playlist and song details are separated from Playlist_Song
3NF	No transitive dependency of non-key attributes	Genre details are stored in Genre and referenced by Song

6.1 Table Creation

The following SQL statements create the tables and establish primary key and foreign key relationships for the Music Streaming Database.

```
CREATE TABLE Music_User (
    User_ID NUMBER PRIMARY KEY,
    User_Name VARCHAR2(50) NOT NULL,
    Email VARCHAR2(100) UNIQUE NOT NULL,
    Join_Date DATE DEFAULT SYSDATE
);

CREATE TABLE Artist (
    Artist_ID NUMBER PRIMARY KEY,
    Artist_Name VARCHAR2(100) NOT NULL,
    Country VARCHAR2(50)
);

CREATE TABLE Album (
    Album_ID NUMBER PRIMARY KEY,
    Album_Title VARCHAR2(100) NOT NULL,
    Release_Date DATE,
    Artist_ID NUMBER,
    FOREIGN KEY (Artist_ID) REFERENCES Artist(Artist_ID)
);

CREATE TABLE Genre (
    Genre_ID NUMBER PRIMARY KEY,
    Genre_Name VARCHAR2(50) UNIQUE NOT NULL
);

CREATE TABLE Song (
    Song_ID NUMBER PRIMARY KEY,
    Song_Title VARCHAR2(100) NOT NULL,
    Duration NUMBER NOT NULL,
    Release_Date DATE,
    Album_ID NUMBER,
    Genre_ID NUMBER,
    FOREIGN KEY (Album_ID) REFERENCES Album(Album_ID),
    FOREIGN KEY (Genre_ID) REFERENCES Genre(Genre_ID)
);

CREATE TABLE Song_Artist (
    Song_ID NUMBER,
```

```
    Artist_ID NUMBER,
    PRIMARY KEY (Song_ID, Artist_ID),
    FOREIGN KEY (Song_ID) REFERENCES Song(Song_ID),
    FOREIGN KEY (Artist_ID) REFERENCES Artist(Artist_ID)
);

CREATE TABLE Playlist (
    Playlist_ID NUMBER PRIMARY KEY,
    Playlist_Name VARCHAR2(100) NOT NULL,
    Created_Date DATE DEFAULT SYSDATE,
    User_ID NUMBER,
    FOREIGN KEY (User_ID) REFERENCES Music_User(User_ID)
);

CREATE TABLE Playlist_Song (
    Playlist_ID NUMBER,
    Song_ID NUMBER,
    Added_Date DATE DEFAULT SYSDATE,
    PRIMARY KEY (Playlist_ID, Song_ID),
    FOREIGN KEY (Playlist_ID) REFERENCES Playlist(Playlist_ID),
    FOREIGN KEY (Song_ID) REFERENCES Song(Song_ID)
);

CREATE TABLE Listening_History (
    History_ID NUMBER PRIMARY KEY,
    User_ID NUMBER,
    Song_ID NUMBER,
    Listen_Time TIMESTAMP DEFAULT SYSTIMESTAMP,
    FOREIGN KEY (User_ID) REFERENCES Music_User(User_ID),
    FOREIGN KEY (Song_ID) REFERENCES Song(Song_ID)
);

CREATE TABLE Song_Like (
    User_ID NUMBER,
    Song_ID NUMBER,
    Like_Date DATE DEFAULT SYSDATE,
    PRIMARY KEY (User_ID, Song_ID),
    FOREIGN KEY (User_ID) REFERENCES Music_User(User_ID),
    FOREIGN KEY (Song_ID) REFERENCES Song(Song_ID)
);

CREATE TABLE Artist_Follow (
    User_ID NUMBER,
    Artist_ID NUMBER,
    Follow_Date DATE DEFAULT SYSDATE,
    PRIMARY KEY (User_ID, Artist_ID),
    FOREIGN KEY (User_ID) REFERENCES Music_User(User_ID),
    FOREIGN KEY (Artist_ID) REFERENCES Artist(Artist_ID)
);
```

Listing 1: Creating Tables for the Music Streaming Database

6.2 Constraints

Constraints maintain data integrity and prevent invalid records from being inserted into the database. The following statements add validation rules to selected tables.

```sql
ALTER TABLE Song
ADD CONSTRAINT chk_song_duration
CHECK (Duration > 0);

ALTER TABLE Music_User
ADD CONSTRAINT chk_user_email
CHECK (Email LIKE '%@%.%');

ALTER TABLE Playlist
ADD CONSTRAINT chk_playlist_name
CHECK (LENGTH(TRIM(Playlist_Name)) > 0);

ALTER TABLE Album
ADD CONSTRAINT chk_album_title
CHECK (LENGTH(TRIM(Album_Title)) > 0);
```

Listing 2: Adding Constraints

6.3 ALTER Operation

The ALTER command modifies the structure of an existing table. In this project, the Language attribute is added to the Song table to store the language of each song.

```sql
ALTER TABLE Song
ADD Language VARCHAR2(30);

ALTER TABLE Song
MODIFY Language VARCHAR2(50);
```

Listing 3: Modifying the Song Table

6.4 DROP Operation

The DROP command permanently removes a database object, including its structure and stored data. The following example creates and removes a temporary table to demonstrate the operation.

```sql
CREATE TABLE Temp_Test (
    Test_ID NUMBER PRIMARY KEY,
    Test_Name VARCHAR2(50)
);

DROP TABLE Temp_Test;
```

Listing 4: Demonstrating the DROP Command

7 SQL DML Commands

Data Manipulation Language (DML) commands are used to insert, modify, and delete records in database tables. In the Music Streaming Database, DML operations manage music catalogue information, user details, playlists, and listening activity.

7.1 INSERT Operation

The INSERT command adds new records to database tables. The following statements insert sample data into the Music Streaming Database.

```
INSERT INTO Music_User
VALUES (1, 'Aarav', 'aarav@gmail.com', DATE '2026-01-10');

INSERT INTO Music_User
VALUES (2, 'Ananya', 'ananya@gmail.com', DATE '2026-02-15');

INSERT INTO Artist
VALUES (101, 'Arijit Singh', 'India');

INSERT INTO Artist
VALUES (102, 'Shreya Ghoshal', 'India');

INSERT INTO Album
VALUES (201, 'Melody Collection', DATE '2025-06-01', 101);

INSERT INTO Genre
VALUES (301, 'Romantic');

INSERT INTO Song
(Song_ID, Song_Title, Duration, Release_Date,
 Album_ID, Genre_ID, Language)
VALUES
(401, 'Tum Hi Ho', 262, DATE '2025-01-01',
 201, 301, 'Hindi');

INSERT INTO Song_Artist
VALUES (401, 101);

INSERT INTO Playlist
VALUES (501, 'My Favorites', SYSDATE, 1);

INSERT INTO Playlist_Song
VALUES (501, 401, SYSDATE);

INSERT INTO Listening_History
VALUES (601, 1, 401, SYSTIMESTAMP);

INSERT INTO Song_Like
VALUES (1, 401, SYSDATE);

INSERT INTO Artist_Follow
VALUES (1, 101, SYSDATE);

COMMIT;
```

Listing 5: Inserting Sample Records

These statements populate the tables with sample users, artists, an album, a genre, a song, a playlist, and user activity. The records are inserted in dependency order to satisfy the foreign key constraints.

7.2 UPDATE Operation

The UPDATE command modifies existing records. The following statements demonstrate updating song information and playlist details.

```
UPDATE Song
SET Song_Title = 'Tum Hi Ho - Remastered'
WHERE Song_ID = 401;

UPDATE Playlist
SET Playlist_Name = 'Romantic Favorites'
WHERE Playlist_ID = 501;

UPDATE Artist
SET Country = 'India'
WHERE Artist_ID = 101;

COMMIT;
```

Listing 6: Updating Existing Records

The first statement updates a song title, the second changes a playlist name, and the third updates an artist's country. The WHERE clause ensures that only the intended records are modified.

7.3 DELETE Operation

The DELETE command removes existing records from a table. The following example removes a song from a playlist without deleting the song itself from the music catalogue.

```
DELETE FROM Playlist_Song
WHERE Playlist_ID = 501
AND Song_ID = 401;

COMMIT;
```

Listing 7: Deleting Existing Records

The statement removes the association between the selected playlist and song. The song remains available in the Song table and can still be included in other playlists. Using a WHERE clause is important to avoid unintentionally deleting all records from a table.

8 SQL Queries

SQL queries are used to retrieve and analyze information stored in the Music Streaming Database. The following queries demonstrate song searching, retrieving related records, calculating listening statistics, identifying popular artists, generating recommendations, and creating views.

8.1 Basic SELECT Queries

The SELECT statement retrieves data from one or more database tables. The following queries display songs, search by song title, and filter songs by language.

```sql
-- Display all songs
SELECT * FROM Song;

-- Search for a song by title
SELECT Song_ID, Song_Title, Duration
FROM Song
WHERE LOWER(Song_Title) LIKE '%tum hi ho%';

-- Display songs in a particular language
SELECT Song_Title, Language
FROM Song
WHERE Language = 'Hindi';

-- Display all available artists
SELECT Artist_ID, Artist_Name, Country
FROM Artist;
```

Listing 8: Basic SELECT Queries

These queries allow users to search the music catalogue and retrieve song and artist information based on specific conditions.

8.2 Joins

Joins combine records from related tables using common attributes. They are useful for displaying songs along with their albums, genres, and associated artists.

```sql
-- Display songs with album and genre details
SELECT S.Song_Title, A.Album_Title, G.Genre_Name
FROM Song S
JOIN Album A ON S.Album_ID = A.Album_ID
JOIN Genre G ON S.Genre_ID = G.Genre_ID;

-- Display songs and their associated artists
SELECT S.Song_Title, AR.Artist_Name
FROM Song S
JOIN Song_Artist SA ON S.Song_ID = SA.Song_ID
JOIN Artist AR ON SA.Artist_ID = AR.Artist_ID;

-- Display playlists with their owners
SELECT P.Playlist_Name, U.User_Name
FROM Playlist P
JOIN Music_User U ON P.User_ID = U.User_ID;
```

Listing 9: Queries Using Joins

The queries retrieve related information from multiple tables without duplicating data. The Song_Artist table supports the association of multiple artists with a song.

8.3 Aggregate Functions

Aggregate functions perform calculations on a group of records. COUNT, AVG, and SUM are used to analyze listening activity and music catalogue information.

```sql
-- Count the total number of songs
SELECT COUNT(*) AS Total_Songs
```

```sql
FROM Song;

-- Calculate the average song duration
SELECT ROUND(AVG(Duration), 2) AS Average_Duration
FROM Song;

-- Count listening events for each song
SELECT S.Song_Title,
       COUNT(LH.History_ID) AS Total_Listens
FROM Song S
LEFT JOIN Listening_History LH
ON S.Song_ID = LH.Song_ID
GROUP BY S.Song_ID, S.Song_Title;

-- Count the number of registered users
SELECT COUNT(*) AS Total_Users
FROM Music_User;
```

Listing 10: Aggregate Function Queries

These queries help determine the size of the music catalogue, average song duration, number of listening events, and total registered users.

8.4 GROUP BY and HAVING

The GROUP BY clause groups records with common values, while HAVING filters the resulting groups based on aggregate conditions.

```sql
-- Find songs played more than once
SELECT S.Song_Title,
       COUNT(LH.History_ID) AS Play_Count
FROM Song S
JOIN Listening_History LH
ON S.Song_ID = LH.Song_ID
GROUP BY S.Song_ID, S.Song_Title
HAVING COUNT(LH.History_ID) > 1;

-- Find users who have listened to more than
-- two songs or listening events
SELECT U.User_Name,
       COUNT(LH.History_ID) AS Total_Listens
FROM Music_User U
JOIN Listening_History LH
ON U.User_ID = LH.User_ID
GROUP BY U.User_ID, U.User_Name
HAVING COUNT(LH.History_ID) > 2;

-- Count songs associated with each genre
SELECT G.Genre_Name,
       COUNT(S.Song_ID) AS Song_Count
FROM Genre G
LEFT JOIN Song S ON G.Genre_ID = S.Genre_ID
GROUP BY G.Genre_ID, G.Genre_Name;
```

Listing 11: GROUP BY and HAVING Queries

These queries identify frequently played songs, users with multiple listening events, and the number of songs available in each genre.

8.5 Subqueries

A subquery is a query nested inside another SQL statement. Subqueries can be used to identify songs that have been played more frequently than average and to recommend songs based on a user's listening history.

```sql
-- Find songs with above-average listening counts
SELECT S.Song_ID, S.Song_Title
FROM Song S
WHERE
    (SELECT COUNT(*)
     FROM Listening_History LH
     WHERE LH.Song_ID = S.Song_ID)
    >
    (SELECT AVG(Play_Count)
     FROM (
         SELECT COUNT(*) AS Play_Count
         FROM Listening_History
         GROUP BY Song_ID
     ));

-- Recommend songs from genres the user has
-- previously listened to, excluding played songs
SELECT DISTINCT S.Song_ID, S.Song_Title
FROM Song S
WHERE S.Genre_ID IN (
    SELECT DISTINCT S2.Genre_ID
    FROM Listening_History LH
    JOIN Song S2 ON LH.Song_ID = S2.Song_ID
    WHERE LH.User_ID = 1
)
AND NOT EXISTS (
    SELECT 1
    FROM Listening_History LH2
    WHERE LH2.User_ID = 1
      AND LH2.Song_ID = S.Song_ID
);

-- Find artists with at least one followed user
SELECT Artist_Name
FROM Artist
WHERE Artist_ID IN (
    SELECT Artist_ID
    FROM Artist_Follow
);
```

Listing 12: Subqueries and Song Recommendations

The first query identifies songs whose listening counts exceed the average among songs with recorded listening events. The second recommends songs from genres previously listened to by user 1 while excluding songs already present in that user's listening history. The recommendation query can be adapted for any user by changing the user ID.

8.6 Views

A view is a virtual table based on a SQL query. Views simplify frequently used queries and provide convenient access to selected information.

```sql
-- Create a view containing song details
CREATE OR REPLACE VIEW Song_Details AS
SELECT S.Song_ID, S.Song_Title, S.Duration,
       A.Album_Title, G.Genre_Name, S.Language
FROM Song S
LEFT JOIN Album A ON S.Album_ID = A.Album_ID
LEFT JOIN Genre G ON S.Genre_ID = G.Genre_ID;

-- Display the view
SELECT * FROM Song_Details;

-- Create a view showing listening statistics
CREATE OR REPLACE VIEW Listening_Statistics AS
SELECT S.Song_ID, S.Song_Title,
       COUNT(LH.History_ID) AS Total_Listens
FROM Song S
LEFT JOIN Listening_History LH
ON S.Song_ID = LH.Song_ID
GROUP BY S.Song_ID, S.Song_Title;

-- Display songs ordered by listening count
SELECT *
FROM Listening_Statistics
ORDER BY Total_Listens DESC;
```

Listing 13: Creating and Using Views

The Song_Details view combines song, album, and genre information. The Listening_Statistics view provides the total number of recorded listening events for each song and can be used to identify popular songs.

9 Transactions and Concurrency

A transaction is a sequence of database operations treated as a single logical unit of work. Transaction management helps maintain data consistency and reliability when modifying records in the Music Streaming Database. Oracle supports transaction control through commands such as COMMIT, ROLLBACK, and SAVEPOINT.

9.1 Transaction Management

Transactions are used to perform related database operations while maintaining data integrity. For example, adding a song to a playlist involves inserting a record into the Playlist_Song table.

```sql
-- Begin a transaction by performing DML operations
INSERT INTO Playlist_Song
    (Playlist_ID, Song_ID, Added_Date)
VALUES
    (501, 401, SYSDATE);

-- Verify the inserted record
SELECT *
FROM Playlist_Song
WHERE Playlist_ID = 501
  AND Song_ID = 401;
```

```
12
13 COMMIT;
```

Listing 14: Transaction Management

Oracle implicitly begins a transaction when a session performs a DML operation. The COMMIT command permanently saves the changes made during the transaction.

9.2 COMMIT, ROLLBACK and SAVEPOINT

COMMIT saves transaction changes permanently, ROLLBACK reverses uncommitted changes, and SAVEPOINT creates a point to which a transaction can be partially rolled back.

```
1  -- Demonstrate SAVEPOINT and ROLLBACK
2  UPDATE Playlist
3  SET Playlist_Name = 'Updated Favorites'
4  WHERE Playlist_ID = 501;
5
6  SAVEPOINT Before_Update;
7
8  UPDATE Playlist
9  SET Playlist_Name = 'Temporary Playlist'
10 WHERE Playlist_ID = 501;
11
12 -- Undo changes made after the savepoint
13 ROLLBACK TO Before_Update;
14
15 -- Save the remaining transaction changes
16 COMMIT;
17
18 -- Demonstrate complete rollback
19 UPDATE Playlist
20 SET Playlist_Name = 'Uncommitted Name'
21 WHERE Playlist_ID = 501;
22
23 ROLLBACK;
```

Listing 15: Transaction Control Commands

The first update changes the playlist name, and a savepoint is created afterward. The second update is undone using ROLLBACK TO, preserving the first update. COMMIT saves the remaining change. The final update is reversed by ROLLBACK because it was not committed.

9.3 Concurrency Control

Concurrency control coordinates simultaneous database operations performed by multiple users. It helps prevent conflicting modifications and maintains data consistency.

Oracle uses transaction isolation and row-level locking mechanisms to manage concurrent access. The following example demonstrates explicitly locking a playlist record before updating it.

```
1  -- Lock the selected playlist record
2  SELECT Playlist_ID, Playlist_Name
3  FROM Playlist
4  WHERE Playlist_ID = 501
5  FOR UPDATE;
6
```

```sql
-- Update the locked record
UPDATE Playlist
SET Playlist_Name = 'Study Playlist'
WHERE Playlist_ID = 501;

-- Release the lock by committing
COMMIT;
```

Listing 16: Concurrency Control Using Row Locking

The FOR UPDATE clause locks the selected row for the current transaction. Other sessions attempting conflicting updates to that row may have to wait until the transaction commits or rolls back. This helps maintain consistency when multiple users access the same data.

10 PL/SQL Implementation

PL/SQL is Oracle's procedural extension to SQL. It supports variables, conditional statements, loops, cursors, and exception handling. In the Music Streaming Database, PL/SQL is used to calculate listening statistics, manage playlists, retrieve song records, and automatically respond to database events.

10.1 Functions

A function is a stored PL/SQL program that returns a value. The following function calculates the number of listening events recorded for a particular song.

```sql
CREATE OR REPLACE FUNCTION Get_Song_Listens(
    p_song_id IN NUMBER
)
RETURN NUMBER
IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM Listening_History
    WHERE Song_ID = p_song_id;

    RETURN v_count;
END;
/

-- Execute the function
SELECT Get_Song_Listens(401) AS Total_Listens
FROM DUAL;
```

Listing 17: Function to Count Song Listens

The function accepts a song ID and returns the total number of listening-history records associated with that song.

10.2 Stored Procedures

A stored procedure is a named PL/SQL block that performs a specific operation. The following procedure adds a song to an existing playlist.

```
CREATE OR REPLACE PROCEDURE Add_Song_To_Playlist(
    p_playlist_id IN NUMBER,
    p_song_id IN NUMBER
)
IS
BEGIN
    INSERT INTO Playlist_Song
        (Playlist_ID, Song_ID, Added_Date)
    VALUES
        (p_playlist_id, p_song_id, SYSDATE);

    DBMS_OUTPUT.PUT_LINE(
        'Song added to playlist successfully.'
    );
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE(
            'This song is already in the playlist.'
        );
    WHEN OTHERS THEN
        RAISE;
END;
/

-- Execute the procedure
SET SERVEROUTPUT ON;

BEGIN
    Add_Song_To_Playlist(501, 401);
    COMMIT;
END;
/
```

Listing 18: Procedure to Add a Song to a Playlist

The procedure inserts a song into the specified playlist. The composite primary key of Playlist_Song prevents the same song from being added to the same playlist more than once. The transaction is committed after the procedure executes successfully.

10.3 Cursors

A cursor processes the result of a query one row at a time. An explicit cursor can be used to retrieve songs and display their titles and durations.

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR song_cursor IS
        SELECT Song_Title, Duration
        FROM Song
        ORDER BY Song_Title;

```

```
    v_title Song.Song_Title%TYPE;
    v_duration Song.Duration%TYPE;
BEGIN
    OPEN song_cursor;

    LOOP
        FETCH song_cursor
        INTO v_title, v_duration;

        EXIT WHEN song_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            v_title || ' - ' ||
            v_duration || ' seconds'
        );
    END LOOP;

    CLOSE song_cursor;
END;
/
```

Listing 19: Cursor to Display Song Details

The cursor retrieves each song record and displays its title and duration. The loop continues until all matching records have been processed.

10.4 Triggers

A trigger is a stored PL/SQL block that executes automatically when a specified database event occurs. The following trigger validates song duration before a record is inserted or updated.

```
CREATE OR REPLACE TRIGGER Validate_Song_Duration
BEFORE INSERT OR UPDATE OF Duration
ON Song
FOR EACH ROW
BEGIN
    IF :NEW.Duration <= 0 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Song duration must be greater than zero.'
        );
    END IF;
END;
/

-- Test the trigger with an invalid duration
-- This statement should raise an error
UPDATE Song
SET Duration = 0
WHERE Song_ID = 401;
```

Listing 20: Trigger to Validate Song Duration

The trigger checks the duration before a song record is inserted or its duration is updated. If the duration is zero or negative, Oracle raises an application error and rejects the operation. This maintains data integrity automatically.

11 File Organization and Indexing

File organization describes how records are stored and accessed in a database, while indexing provides additional structures to speed up data retrieval. In the Music Streaming Database, efficient access is important for searching songs, retrieving artist information, and analyzing listening history.

11.1 File Organization

Oracle Database manages the physical storage of table data using data blocks, extents, and segments. The Music Streaming Database stores information in relational tables, with primary keys and foreign keys maintaining relationships between records.

For example, the Song table stores song details, while the Listening_History table records individual listening events. Queries can retrieve related information using primary keys, foreign keys, and joins. Appropriate indexes can reduce the number of data blocks that Oracle must examine when searching for specific records.

11.2 ISAM

Indexed Sequential Access Method (ISAM) is a file organization technique that maintains records in an ordered structure with an index to support direct and sequential access. It can be useful when records are accessed frequently using a search key.

In a music streaming system, an indexed sequential organization could support searches using song identifiers or other frequently accessed attributes. However, ISAM is a database file organization concept and is not the same as Oracle's standard B-tree indexing mechanism.

11.3 B+ Tree Indexing

A B+ tree is a balanced tree structure commonly used for database indexes. It supports efficient equality searches, range queries, and ordered retrieval. Oracle uses B-tree indexes by default for many ordinary index definitions.

The following statements demonstrate creating indexes on columns frequently used in song searches and listening-history analysis.

```
1  -- Index for searching songs by title
2  CREATE INDEX idx_song_title
3  ON Song(Song_Title);
4
5  -- Index for retrieving listening history by user
6  CREATE INDEX idx_history_user
7  ON Listening_History(User_ID);
8
9  -- Composite index for user listening-history queries
10 CREATE INDEX idx_history_user_song
11 ON Listening_History(User_ID, Song_ID);
12
13 -- Example query using indexed columns
14 SELECT Song_ID, Song_Title
15 FROM Song
16 WHERE Song_Title = 'Tum Hi Ho';
17
```

```sql
SELECT Song_ID, Listen_Time
FROM Listening_History
WHERE User_ID = 1
  AND Song_ID = 401;
```

Listing 21: Creating Indexes

The indexes can improve retrieval when the indexed columns are used in suitable query conditions. The composite index on Listening_History supports queries that filter by User_ID and may also support queries filtering by User_ID alone. Indexes require additional storage and may increase the cost of insert, update, and delete operations, so they should be created based on actual query requirements.

12 Testing

Testing is performed to verify that the Music Streaming Database operates correctly and maintains data integrity. The tests cover table operations, song searches, playlist management, listening statistics, and database constraints. Expected results are compared with actual results after executing the corresponding SQL statements.

12.1 Test Cases

The following test cases cover the main functionalities of the database.

Table 12: Test Cases for the Music Streaming Database

No.	Test Case	Operation	Expected Result
1	Insert user	Insert a valid user record	User record is inserted
2	Search song	Search using a song title	Matching songs are displayed
3	Retrieve album details	Join Song and Album tables	Related album details are displayed
4	Create playlist	Insert a playlist for an existing user	Playlist is created successfully
5	Add song to playlist	Insert a valid playlist-song association	Association is stored
6	Record listening event	Insert a valid listening-history record	Listening event is recorded
7	Calculate listening count	Count history records for a song	Total listening events are returned
8	Recommend songs	Execute the listening-history recommendation query	Eligible songs from previously listened genres are returned

These test cases verify the database's main operations, including catalogue search, playlist management, listening-history analysis, and recommendations.

12.2 Normal and Boundary Cases

Normal cases verify that valid inputs are processed correctly. Boundary cases test values near the limits imposed by the schema and constraints.

Table 13: Normal and Boundary Test Cases

Test	**Input or Condition**	**Expected Result**
Valid song duration	Duration greater than zero	Song record is accepted
Minimum valid duration	Duration equals 1	Song record is accepted
Long song title	Title within the column limit	Song record is accepted
Empty search result	Search for a nonexistent title	No matching records are returned
Repeated listening	Same user listens to the same song again	A separate history record is stored with a new History_ID
Repeated playlist entry	Add the same song to the same playlist twice	Duplicate association is rejected by the composite primary key

The tests confirm that valid records can be stored, repeated listening events can be recorded independently, and duplicate playlist-song associations are prevented.

12.3 Invalid Input Testing

Invalid input testing checks whether the database rejects records that violate constraints or reference nonexistent records.

Table 14: Invalid Input Test Cases

Test	**Invalid Input**	**Expected Result**
Duplicate user ID	Insert an existing User_ID	Primary key violation
Duplicate email	Insert an email already used by another user	Unique constraint violation
Invalid song duration	Insert or update a song with Duration = 0	Check constraint or trigger rejects the operation
Invalid foreign key	Add a playlist for a nonexistent user	Foreign key violation
Duplicate playlist entry	Insert an existing Playlist_ID and Song_ID combination	Composite primary key violation
Missing required value	Insert a user without a required name or email	NOT NULL constraint violation

Invalid input testing demonstrates that primary keys, foreign keys, unique constraints, NOT NULL constraints, and duration validation help preserve data integrity. The expected outcomes should be verified by executing the test statements in Oracle.

13 Reports

Reports provide summarized information about music catalogue details and user activity. The Music Streaming Database generates reports on artists and their songs, popular songs based on listening history, genre-wise song distribution, and user listening activity. These reports help analyze music preferences and usage patterns.

13.1 Artist Report

The artist report displays artists along with their albums and associated songs. It can be used to examine the music catalogue associated with each artist.

```
SELECT AR.Artist_ID,
       AR.Artist_Name,
       AL.Album_Title,
       S.Song_Title
FROM Artist AR
LEFT JOIN Album AL
ON AR.Artist_ID = AL.Artist_ID
LEFT JOIN Song_Artist SA
ON AR.Artist_ID = SA.Artist_ID
LEFT JOIN Song S
ON SA.Song_ID = S.Song_ID
ORDER BY AR.Artist_Name, AL.Album_Title;
```

Listing 22: Artist Report

This report presents artist names, their albums, and their associated songs. Artists without matching albums or songs can also appear because LEFT JOIN is used.

13.2 Popular Songs Report

The popular songs report ranks songs according to the number of recorded listening events.

```
SELECT S.Song_ID,
       S.Song_Title,
       COUNT(LH.History_ID) AS Total_Listens
FROM Song S
LEFT JOIN Listening_History LH
ON S.Song_ID = LH.Song_ID
GROUP BY S.Song_ID, S.Song_Title
ORDER BY Total_Listens DESC, S.Song_Title;
```

Listing 23: Popular Songs Report

The report displays songs in descending order of listening count. Songs with no recorded listening events are included with a count of zero.

13.3 Genre-Wise Report

The genre-wise report summarizes the number of songs available in each genre.

```
SELECT G.Genre_Name,
       COUNT(S.Song_ID) AS Total_Songs
FROM Genre G
```

```
LEFT JOIN Song S
ON G.Genre_ID = S.Genre_ID
GROUP BY G.Genre_ID, G.Genre_Name
ORDER BY Total_Songs DESC;
```

Listing 24: Genre-Wise Song Report

This report helps identify genres with larger song collections. The LEFT JOIN also includes genres that do not currently contain any songs.

13.4 User Listening Statistics

The user listening statistics report shows the total number of listening events recorded for each user.

```
SELECT U.User_ID,
       U.User_Name,
       COUNT(LH.History_ID) AS Total_Listens
FROM Music_User U
LEFT JOIN Listening_History LH
ON U.User_ID = LH.User_ID
GROUP BY U.User_ID, U.User_Name
ORDER BY Total_Listens DESC, U.User_Name;
```

Listing 25: User Listening Statistics

This report summarizes user activity and allows users to be compared based on their recorded listening events. Each history record is counted as a separate listening event, even when the same song is played multiple times.

14 Results and Discussion

The Music Streaming Database is designed to organize and manage music catalogue information and user activities using Oracle SQL and PL/SQL. The relational schema establishes connections between users, artists, albums, songs, genres, playlists, and listening history through primary key and foreign key constraints.

The SQL operations support inserting, updating, and deleting records while maintaining referential integrity. SELECT queries and joins retrieve song details, album information, artist details, and playlist contents. Aggregate functions and GROUP BY clauses help calculate listening counts, identify popular songs, and generate genre-wise statistics.

The database also supports listening-history-based song recommendations. By analyzing the genres of songs previously played by a user, SQL queries can identify other songs belonging to similar genres while excluding songs already listened to by that user.

PL/SQL functions, procedures, cursors, and triggers demonstrate how database operations can be implemented and controlled within Oracle. Indexes on frequently searched song titles and listening-history attributes can also improve the efficiency of relevant queries.

Overall, the project demonstrates how relational database concepts can be applied to a music streaming environment. Its design supports structured data management, music discovery, playlist organization, and listening activity analysis. Actual performance and correctness depend on executing the SQL and PL/SQL statements and validating them with suitable test data.

15 Conclusion

The Music Streaming Database Management System demonstrates the application of relational database concepts to manage music-related information and user activities. The system organizes users, artists, albums, songs, genres, playlists, and listening history into related tables using primary keys, foreign keys, and integrity constraints.

The project incorporates SQL operations such as data insertion, updating, deletion, searching, joins, aggregate functions, subqueries, and views. It also demonstrates transaction management through COMMIT, ROLLBACK, and SAVEPOINT. PL/SQL functions, stored procedures, cursors, and triggers provide additional support for database operations and validation.

The database supports playlist management, artist-related reports, listening statistics, and SQL-based song recommendations derived from users' listening history and preferred genres. Normalization helps reduce data redundancy, while indexing can improve the efficiency of frequently executed queries.

In conclusion, the project provides a structured foundation for a music streaming database and demonstrates the practical use of Oracle SQL and PL/SQL in database design, implementation, and information retrieval.

16 SDG Mapping

The Music Streaming Database Management System contributes to selected United Nations Sustainable Development Goals (SDGs) through the application of database technology, organized information management, and efficient access to digital music resources.

Table 15: Mapping of the Project to Sustainable Development Goals

SDG	**Goal**	**Project Contribution**
SDG 4	Quality Education	The project provides practical learning in database design, SQL, PL/SQL, normalization, and query processing.
SDG 9	Industry, Innovation and Infrastructure	The project demonstrates the use of relational database technology to organize music catalogue information and user activity records.
SDG 12	Responsible Consumption and Production	Structured data management reduces unnecessary duplication of stored information through relational design and normalization.

The project is most directly related to SDG 4 and SDG 9 because it applies database technologies to an educational project and demonstrates structured information management. Its connection to SDG 12 is indirect, through the use of normalization and efficient data organization.

17 Future Scope

The Music Streaming Database can be extended with additional features to improve music discovery, user interaction, and data analysis. The following enhancements can be considered in future development.

- **Advanced Song Recommendations:** Recommendation queries can be improved by considering listening frequency, liked songs, followed artists, and similarities between users' music preferences.
- **Enhanced Search:** Search functionality can be extended to support partial song titles, artist names, albums, genres, and language-based filtering.
- **Listening Trend Analysis:** Additional reports can identify frequently played songs, popular genres, most-listened-to artists, and changes in listening patterns over time.
- **User Interface Integration:** The database can be connected to a web or mobile application to provide an interface for searching songs, creating playlists, and viewing listening history.
- **Performance Optimization:** Indexes, query optimization, and suitable database design improvements can be applied as the volume of songs and listening records increases.
- **Improved Data Validation:** Additional constraints, triggers, and PL/SQL procedures can be introduced to strengthen data integrity and handle invalid inputs consistently.

These enhancements can extend the database into a more comprehensive music management system while maintaining data consistency, efficient retrieval, and support for personalized music discovery.

Bibliography

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson, 2016.

[3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd Edition, McGraw-Hill, 2003.

[4] Oracle Corporation, *Oracle Database SQL Language Reference*, Oracle Database Documentation. Available at: https://docs.oracle.com/en/database/

[5] Oracle Corporation, *Oracle Database PL/SQL Language Reference*, Oracle Database Documentation. Available at: https://docs.oracle.com/en/database/

[6] W3Schools, *SQL Tutorial*, W3Schools Online Web Tutorials. Available at: https://www.w3schools.com/sql/